Time-Tracking Pipeline Project Proposal

Executive Summary

This proposal presents a data pipeline project to streamline data collection and processing on time-tracking datasets. The pipeline will help determine team bandwidth and throughput, enhancing decision-making and operational efficiency. It aims to provide a scalable solution that integrates data sources, ensures data quality, and delivers actionable insights from the data presented in the web app.

Introduction

Data pipelines are crucial for converting raw data into meaningful insights. However, real-world data often comes in various formats and contains null values and incomplete entries that can disrupt data analysis. This is also true for the time-tracking dataset, which I have identified as having several issues that need to be resolved before it can be loaded into the database for further throughput and bandwidth analysis.

Objectives

The primary objectives of this data pipeline project are:

- Identify issues in the dataset and plan the data transformation and cleaning processes using ETL methods.
- Establish a reliable and scalable data pipeline to streamline data collection, transformation, and loading processes.
- Enable data visualization on the web app for tracking team members and project time in table form.
- Provide a flexible framework that can easily adapt to new data sources and changing business requirements.

Data Pipeline Architecture

The diagram below illustrates the architecture of this data pipeline.

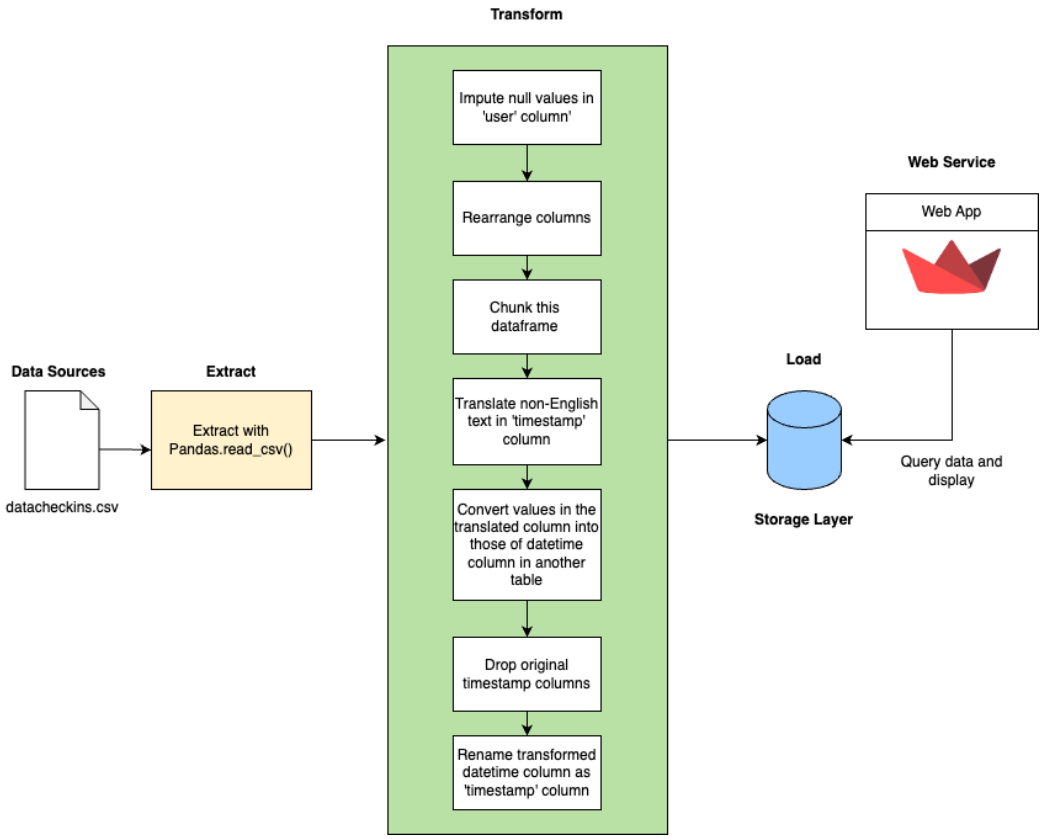


Figure 1: The architecture of time-tracking pipeline

Data Source

In this project, the sole data source is a .csv file containing time-tracking information, consisting of 20,500 rows and 4 columns: user, hours, project, and timestamp. During the data exploration phase, I identified several concerns that necessitate ETL processes to address issues in two specific columns: user and timestamp.

User column:

This column contains five null values, which may present challenges for subsequent data analysis. Therefore, it is imperative to impute these null values.

Timestamp column:

This column poses challenges due to the presence of various string datetime values. Additionally, some rows include non-English characters, specifically Ukrainian and Russian Cyrillic scripts.

Despite column issues, the column name alignment is not user-friendly as illustrated below:

	user	timestamp	hours	project
15797	NaN	2017-12-27 10:36:14.000121 UTC	4.00	project-40
15798	NaN	2017-12-27 10:36:14.000121 UTC	3.00	learning
17572	NaN	2017-10-12 10:31:44.000227 UTC	2.75	project-47
17573	NaN	2017-10-12 10:31:44.000227 UTC	4.00	bizdev
17574	NaN	2017-10-12 10:31:44.000227 UTC	1.00	transit

Figure 2: An example of time-tracking dataset with improper column alignment

To enhance the comprehensibility of the dataset, it is recommended to rearrange the columns in the following order: user, project, hours, and timestamp.

Data Extraction (Extract)

The pandas library is employed as an efficient solution for this stage. The CSV file can be seamlessly converted into a data frame, facilitating subsequent data cleaning and transformation processes.

Data Transformation (Transform)

The following steps are listed and implemented in the code to resolve issues in the Data Source section.

1. Impute null values in user column with a default string value: ‘unknown user’
2. Rearrange the column names
3. Translate non-English text in timestamp column into English text using langdetect and deep_translator Python library
4. Convert values in a translated timestamp column into another different column with datetime type
5. Drop original timestamp column that contains non-English text values
6. Rename the newly created datetime column as timestamp column, the same name as that of the original dataset.

Data Load (Load)

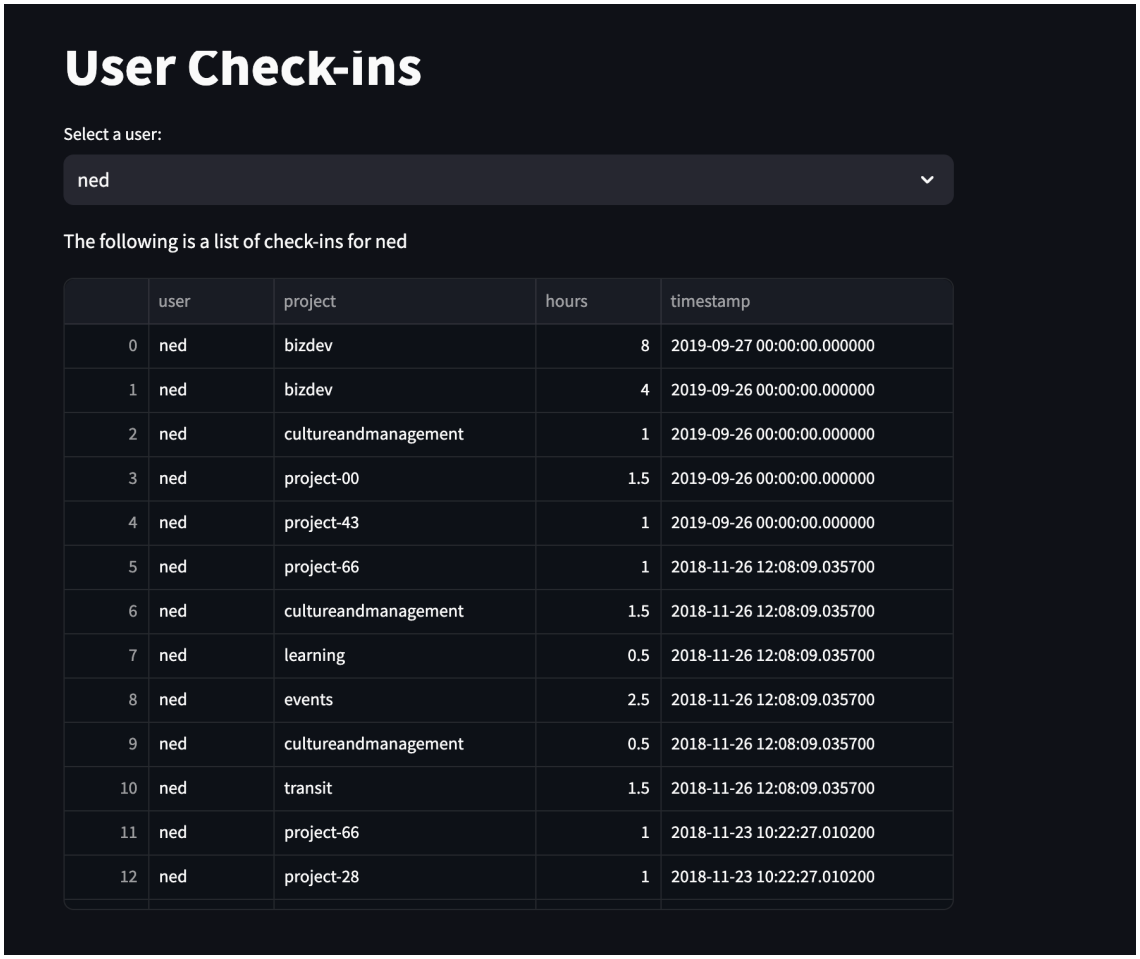
In this project, I use ‘sqlalchemy’ as a python library to create a database file, make a database connection to create a table, insert records of cleaned and transformed check-

in data. The chosen database is SQlite for this small-scale storage layer due to its simple configuration and seamless integration with ‘Streamlit’ web app.

Web Service

The web app is deployed as a ‘Streamlit’ web app. I chose this python framework due to its easy and fast web app deployment feature, which enables me to deploy Python code and launch a public web app instantly. The URL for this project can be found here.

In this initial solution, the data stored within the specified database is accessed through a user selected via the drop-down menu on the web service. The resulting display will show a list of all check-ins associated with a particular user. The table functionality includes the ability to sort entries in ascending or descending order based on project name, hours, and timestamp, thereby enhancing the interactivity of the user experience.



	user	project	hours	timestamp
0	ned	bizdev	8	2019-09-27 00:00:00.000000
1	ned	bizdev	4	2019-09-26 00:00:00.000000
2	ned	cultureandmanagement	1	2019-09-26 00:00:00.000000
3	ned	project-00	1.5	2019-09-26 00:00:00.000000
4	ned	project-43	1	2019-09-26 00:00:00.000000
5	ned	project-66	1	2018-11-26 12:08:09.035700
6	ned	cultureandmanagement	1.5	2018-11-26 12:08:09.035700
7	ned	learning	0.5	2018-11-26 12:08:09.035700
8	ned	events	2.5	2018-11-26 12:08:09.035700
9	ned	cultureandmanagement	0.5	2018-11-26 12:08:09.035700
10	ned	transit	1.5	2018-11-26 12:08:09.035700
11	ned	project-66	1	2018-11-23 10:22:27.010200
12	ned	project-28	1	2018-11-23 10:22:27.010200

Figure 3: The example web app with a user named ‘ned’ and his check-in records

Risks and Mitigation

Potential risks and mitigation strategies include:

Scalability:

With limited processing power on my laptop, the translation process takes approximately 7-10 minutes. If the number of records is significantly higher than this dataset, it could lead to resource throttling and eventual pipeline failure.

A solution is to define a chunk size to break down the large dataset into smaller data frames of equal size (in this case, 500). This approach gives the translator function adequate time to process translations for non-English text effectively, which often fails when dealing with a single 20,500-row data frame.

A long-term solution could involve migrating this data pipeline to a cloud service. This would leverage features like auto-scaling or serverless architecture to provide flexibility in processing the dataset and ensure better performance.

Null value on user:

As previously noted, there are five rows with null values in the 'user' column. This issue could pose problems as the records are incomplete.

An interim solution would be to impute this null value with a placeholder user named 'unknown user'. However, for a long-term resolution, data validation should be implemented at the source that generates this CSV file to prevent such errors from occurring.

Conclusion

In summary, the time-tracking dataset contains several data issues that need proper transformation and processing after data exploration. Various tools are used to streamline these processes, load the cleaned data into a database, and display processed data on the public website for users to reference for bandwidth and throughput analysis. This solution is an initial version of the pipeline that will require further improvements based on additional requirements and feedback.